

model-training

May 19, 2026

```
[1]: # pyright: reportUnusedImport=false, reportMissingImports=false,
      ↪reportMissingModuleSource=false
import pandas as pd
import numpy as np
import tensorflow as tf
import os
import seaborn as sb
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten,
      ↪Dropout, Rescaling
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split
```

```
2026-05-19 01:06:58.756753: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not
find cuda drivers on your machine, GPU will not be used.
2026-05-19 01:06:59.426439: E
tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.cc:9342] Unable to
register cuDNN factory: Attempting to register factory for plugin cuDNN when one
has already been registered
2026-05-19 01:06:59.426545: E
tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.cc:609] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
2026-05-19 01:06:59.429220: E
tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:1518] Unable to
register cuBLAS factory: Attempting to register factory for plugin cuBLAS when
one has already been registered
2026-05-19 01:06:59.759843: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not
find cuda drivers on your machine, GPU will not be used.
2026-05-19 01:06:59.763120: I tensorflow/core/platform/cpu_feature_guard.cc:182]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
2026-05-19 01:07:02.411046: W
tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not
```

find TensorRT

```
[2]: normal_cells=os.listdir('brain_tumor_dataset/no/')
      tumor_cells=os.listdir('brain_tumor_dataset/yes/')
```

1 total images in each folder

```
[3]: print('Number of normal cells: ',len(normal_cells))
      print('Number of tumor cells: ',len(tumor_cells))
```

Number of normal cells: 98

Number of tumor cells: 157

```
[4]: TRAIN_DIR='brain_tumor_dataset'
```

2 image resize

```
[5]: # Instantiate the dataset
      train_dataset = tf.keras.utils.image_dataset_from_directory(
          TRAIN_DIR,
          image_size=(128, 128),
          batch_size=32,
          label_mode='binary',
          seed=100
      )

      # Check the type
      dataset_type = type(train_dataset)
      print(f'train_dataset inherits from tf.data.Dataset: {issubclass(dataset_type,
          ↪tf.data.Dataset)}')
```

Found 255 files belonging to 2 classes.

train_dataset inherits from tf.data.Dataset: True

```
[6]: val_ds = tf.keras.utils.image_dataset_from_directory(
      TRAIN_DIR,
      validation_split=0.2,
      subset='validation',
      image_size=(128, 128),
      batch_size=32,
      label_mode='binary',
      seed=100
  )
  dataset_type = type(train_dataset)
  print(f'train_dataset inherits from tf.data.Dataset: {issubclass(dataset_type,
      ↪tf.data.Dataset)}')
```

```
Found 255 files belonging to 2 classes.
Using 51 files for validation.
Using 51 files for validation.
train_dataset inherits from tf.data.Dataset: True
```

3 validating the image shapes

```
[7]: # check the image shape and labels
for images, labels in train_dataset.take(1):
    print(f'Image batch shape: {images.shape}')
    print(f'Label batch shape: {labels.shape}')
```

```
Image batch shape: (32, 128, 128, 3)
Label batch shape: (32, 1)
```

```
[8]: # generate augmented images
data_augmentation=ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    vertical_flip=True,
    horizontal_flip=True
)
```

```
[9]: rescale=Rescaling(1./255)
train_dataset = train_dataset.map(lambda x, y: (rescale(x), y))
val_ds = val_ds.map(lambda x, y: (rescale(x), y))
```

4 improving performance

```
[10]: train_dataset=(
    train_dataset.cache()
    .shuffle(100).
    prefetch(buffer_size=tf.data.AUTOTUNE)
)
```

```
[11]: val_ds=(
    val_ds.cache()
    .prefetch(buffer_size=tf.data.AUTOTUNE)
)
```

5 defining early stop at 82 percent

```
[12]: class EarlyStopping(tf.keras.callbacks.Callback):
        def on_epoch_end(self, epoch, logs=None):
            if logs.get("accuracy") > 0.82:
                print("\nReached 82% accuracy so cancelling training!")
                self.model.stop_training = True
```

```
[13]: model = Sequential([
        tf.keras.layers.Input(shape=(128, 128, 3)),
        Conv2D(16, 3, activation='relu'),
        MaxPooling2D(2, 2),
        Conv2D(32, 3, activation='relu'),
        MaxPooling2D(2, 2),
        Conv2D(64, 3, activation='relu'),
        MaxPooling2D(2, 2),
        Conv2D(128, 3, activation='relu'),
        MaxPooling2D(2, 2),

        tf.keras.layers.Flatten(),
        Dense(256, activation='relu'),
        Dense(1, activation='sigmoid')
    ])
```

```
[14]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 16)	448
max_pooling2d (MaxPooling2D)	(None, 63, 63, 16)	0
conv2d_1 (Conv2D)	(None, 61, 61, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 32)	0
conv2d_2 (Conv2D)	(None, 28, 28, 64)	18496

max_pooling2d_2 (MaxPoolin g2D)	(None, 14, 14, 64)	0
conv2d_3 (Conv2D)	(None, 12, 12, 128)	73856
max_pooling2d_3 (MaxPoolin g2D)	(None, 6, 6, 128)	0
flatten (Flatten)	(None, 4608)	0
dense (Dense)	(None, 256)	1179904
dense_1 (Dense)	(None, 1)	257

```
=====
Total params: 1277601 (4.87 MB)
Trainable params: 1277601 (4.87 MB)
Non-trainable params: 0 (0.00 Byte)
-----
```

```
[15]: model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss=tf.keras.losses.BinaryCrossentropy(from_logits=False),
        metrics=['accuracy']
    )
```

6 using pre-build early stopping

using validation set also for fitting

```
[16]: history = model.fit(
        train_dataset,
        validation_data=val_ds,
        epochs=10,
        callbacks=[
            tf.keras.callbacks.EarlyStopping(
                monitor='val_accuracy',
                patience=5,
                restore_best_weights=True
            )
        ]
    )
```

Epoch 1/10

```
8/8 [=====] - 5s 321ms/step - loss: 0.6236 - accuracy:
0.6667 - val_loss: 0.5472 - val_accuracy: 0.7647
```

```

Epoch 2/10
8/8 [=====] - 2s 295ms/step - loss: 0.5502 - accuracy:
0.7490 - val_loss: 0.4599 - val_accuracy: 0.8235
Epoch 3/10
8/8 [=====] - 2s 270ms/step - loss: 0.5147 - accuracy:
0.7804 - val_loss: 0.4200 - val_accuracy: 0.8431
Epoch 4/10
8/8 [=====] - 2s 288ms/step - loss: 0.4695 - accuracy:
0.7922 - val_loss: 0.3640 - val_accuracy: 0.8627
Epoch 5/10
8/8 [=====] - 2s 267ms/step - loss: 0.4362 - accuracy:
0.8118 - val_loss: 0.3180 - val_accuracy: 0.8824
Epoch 6/10
8/8 [=====] - 2s 296ms/step - loss: 0.4077 - accuracy:
0.8157 - val_loss: 0.3132 - val_accuracy: 0.9020
Epoch 7/10
8/8 [=====] - 2s 269ms/step - loss: 0.3754 - accuracy:
0.8314 - val_loss: 0.2723 - val_accuracy: 0.9020
Epoch 8/10
8/8 [=====] - 2s 287ms/step - loss: 0.3043 - accuracy:
0.8627 - val_loss: 0.1866 - val_accuracy: 0.9412
Epoch 9/10
8/8 [=====] - 2s 291ms/step - loss: 0.2409 - accuracy:
0.9098 - val_loss: 0.1569 - val_accuracy: 0.9804
Epoch 10/10
8/8 [=====] - 2s 306ms/step - loss: 0.1980 - accuracy:
0.9294 - val_loss: 0.1041 - val_accuracy: 0.9804

```

```

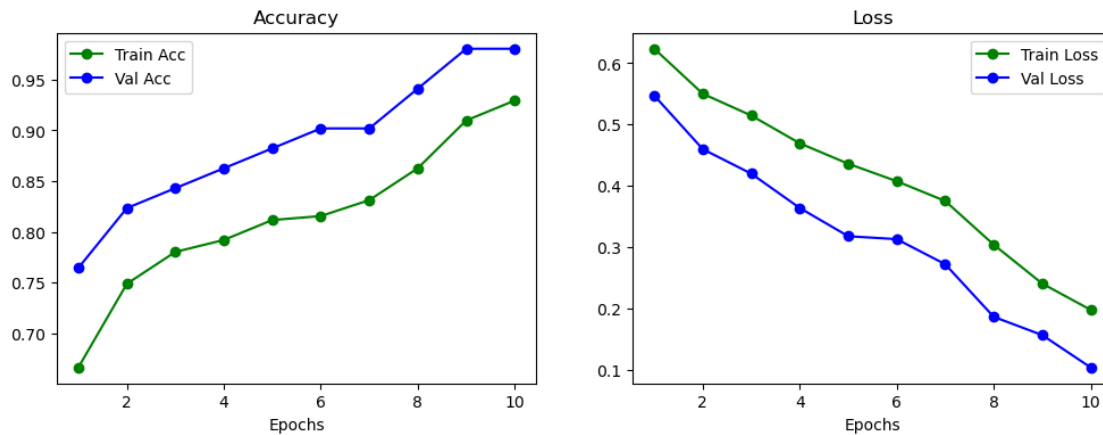
[17]: import matplotlib.pyplot as plt
try:
    h = history.history if hasattr(history, 'history') else history
    epochs = range(1, len(next(iter(h.values())))) + 1
    plt.figure(figsize=(12,4))
    plt.subplot(1,2,1)
    if 'accuracy' in h or 'acc' in h:
        acc = h.get('accuracy') or h.get('acc')
        plt.plot(epochs, acc, 'go-', label='Train Acc')
    if 'val_accuracy' in h or 'val_acc' in h:
        val_acc = h.get('val_accuracy') or h.get('val_acc')
        plt.plot(epochs, val_acc, 'bo-', label='Val Acc')
    plt.title('Accuracy')
    plt.xlabel('Epochs')
    plt.legend()
    plt.subplot(1,2,2)
    if 'loss' in h:
        plt.plot(epochs, h['loss'], 'go-', label='Train Loss')
    if 'val_loss' in h:

```

```

plt.plot(epochs, h['val_loss'], 'bo-', label='Val Loss')
plt.title('Loss')
plt.xlabel('Epochs')
plt.legend()
plt.show()
except Exception as e:
    print('Plot failed:', e)
    print('Ensure `history` is the return value from `model.fit(...)`')

```



```
[19]: Test_Dir="test_images"
```

```
[20]: test_dataset = tf.keras.utils.image_dataset_from_directory(
    Test_Dir,
    image_size=(128, 128),
    batch_size=32,
    label_mode='binary',
    seed=100
)
```

Found 253 files belonging to 2 classes.

```
[21]: model.evaluate(test_dataset)
```

```
8/8 [=====] - 1s 48ms/step - loss: 16.0066 - accuracy: 0.9368
```

```
[21]: [16.006555557250977, 0.9367588758468628]
```

```
[22]: model.predict(test_dataset)
```

```
8/8 [=====] - 1s 45ms/step
```

```
[22]: array([[0.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
            [0.0000000e+00],
            [1.0000000e+00],
            [1.0000000e+00],
            [0.0000000e+00],
```


[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[9.9999964e-01],
[5.7981946e-02],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[6.4949235e-12],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],

[illegible]

[illegible]

[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[7.8042122e-17],
[0.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],

```

[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00],
[1.0000000e+00],
[1.0000000e+00],
[0.0000000e+00]]], dtype=float32)

```

6.1 Confusion Matrix

```

[24]: from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc
      ↪ auc
import matplotlib.pyplot as plt
import seaborn as sns

# Get predictions on test set
y_true = []
y_pred_prob = []

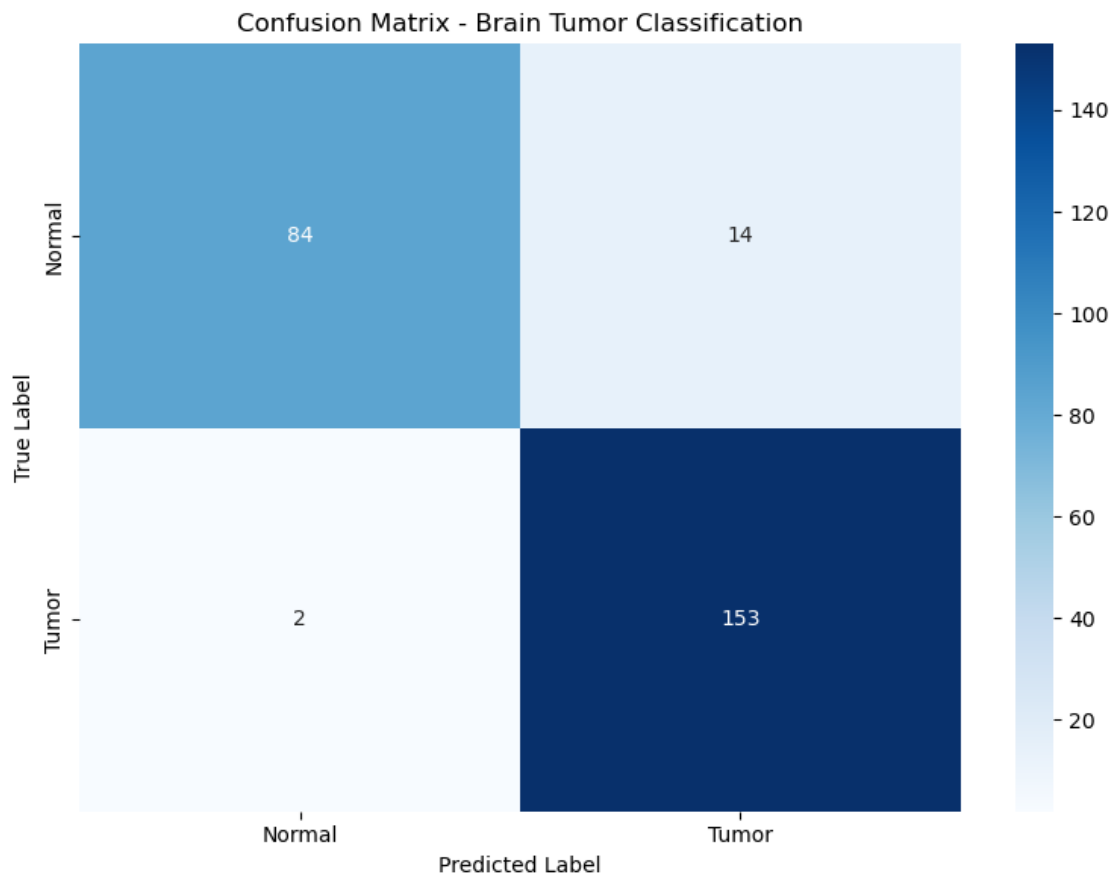
for images, labels in test_dataset:
    y_true.extend(labels.numpy())
    predictions = model.predict(images, verbose=0)
    y_pred_prob.extend(predictions.flatten())

y_true = np.array(y_true)
y_pred = (np.array(y_pred_prob) >= 0.5).astype(int)

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Tumor'],
            yticklabels=['Normal', 'Tumor'])
plt.title('Confusion Matrix - Brain Tumor Classification')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')

```

```
plt.tight_layout()
plt.show()
```



6.2 Decision Boundary Visualization (2D Projection)

```
[25]: # Extract features from second-to-last layer and visualize decision boundary
from sklearn.decomposition import PCA

# Create a model that outputs features from before the final classification
# layer
feature_extractor = tf.keras.Model(
    inputs=model.input,
    outputs=model.layers[-2].output # Get Dense(256) layer output
)

# Extract features and predictions
features_list = []
labels_list = []
```

```

for images, labels in test_dataset:
    features = feature_extractor.predict(images, verbose=0)
    features_list.append(features)
    labels_list.extend(labels.numpy())

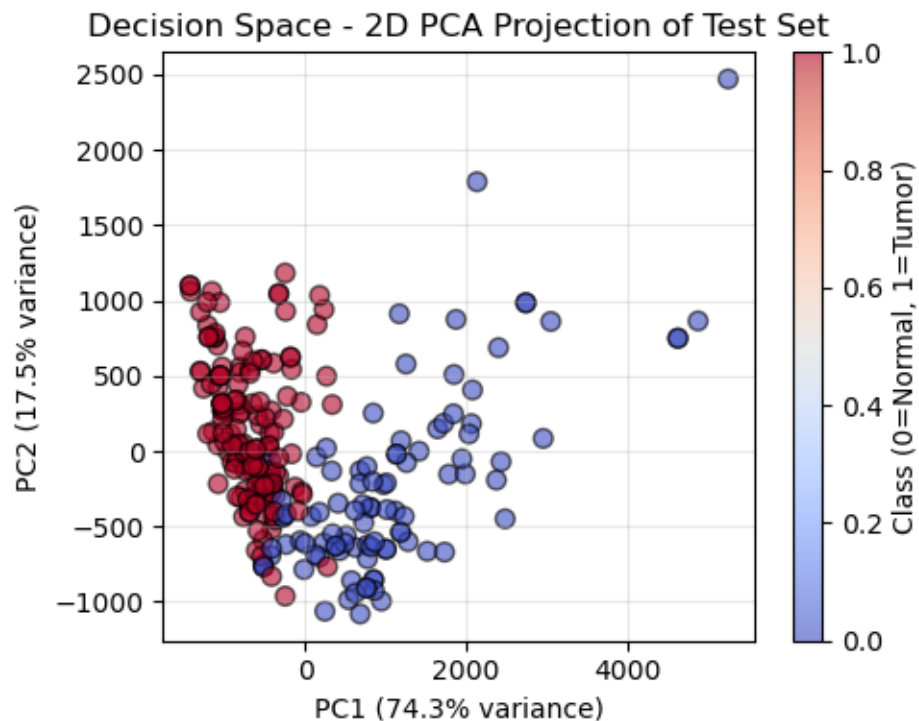
features = np.vstack(features_list)
labels = np.array(labels_list)

# Reduce to 2D using PCA
pca = PCA(n_components=2)
features_2d = pca.fit_transform(features)

# Plot decision boundary and data points
plt.figure(figsize=(5, 4))
scatter = plt.scatter(features_2d[:, 0], features_2d[:, 1],
                      c=labels, cmap='coolwarm', s=50, alpha=0.6, edgecolors='k')
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')
plt.title('Decision Space - 2D PCA Projection of Test Set')
plt.colorbar(scatter, label='Class (0=Normal, 1=Tumor)')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"PCA explained variance: {pca.explained_variance_ratio_.sum():.1%}")

```



PCA explained variance: 91.8%

7 upload images

```
[27]: import ipywidgets as widgets
from IPython.display import display, clear_output
import io
from PIL import Image

uploader = widgets.FileUpload(
    accept='image/*',    # Accept all image types
    multiple=False,      # Single file upload
    description='Upload Image',
    button_style='info'  # Blue color
)
output = widgets.Output()

def on_predict_clicked(b):
    with output:
        clear_output()

        if not uploader.value:
            print(" Please upload an image first.")
            return

        print("Processing...")

        # Support both dict (new ipywidgets) and list/tuple formats
        if isinstance(uploader.value, dict):
            uploaded_file = next(iter(uploader.value.values()))
        else:
            uploaded_file = uploader.value[0]

        # filename may be at top-level or under metadata depending on widget/
        ↪version
        filename = uploaded_file.get('name') or (uploaded_file.get('metadata') or
        ↪or {}).get('name', 'uploaded_image')
        content = uploaded_file.get('content') or uploaded_file.get('data')

        image = Image.open(io.BytesIO(content)).convert('RGB')
```



```

display_img = image.copy()
display_img.thumbnail((200, 200))

image = image.resize((128, 128))
image_array = np.array(image) / 255.0
image_array = np.expand_dims(image_array, axis=0)

prediction = model.predict(image_array, verbose=0)[0][0]
label = 'Tumor' if prediction >= 0.5 else 'Normal'
confidence = prediction if prediction >= 0.5 else 1 - prediction

display(display_img)
print(f'File: {filename}')
print(f'Prediction: {label}')
print(f'Confidence: {confidence:.2%}')

predict_btn = widgets.Button(
    description='Predict Tumor',
    button_style='success',
    tooltip='Click to analyze the uploaded image'
)
predict_btn.on_click(on_predict_clicked)

print("--- Brain Tumor Classifier ---")
display(widgets.VBox([uploader, predict_btn, output]))

```

--- Brain Tumor Classifier ---

```

VBox(children=(FileUpload(value=(), accept='image/*', button_style='info',
    description='Upload Image'), Button...

```

7.0.1 wrapping our model in .keras(.h5) to consume at backend fast api

```
[52]: model.save('brain_tumor_classifier.keras')
```